Spatial Data Analysis in R

Ina Krapp

2025-10-15

```
library(sf) # Spatial Features - vector data
library(tidyverse)
library(here)
library(spatstat) # Point Pattern Analysis
library(terra) # For robust spatial operations, especially with raster data
library(mgcv) # Generalized additive regression
```

1. Introduction

The spatial data ecosystem in R comprises a number of packages, each tailored to a particular data type or workflow.
The most common data objects are:

- **Points** – e.g. locations of cities or sampling sites
- **Lines** – e.g. roads, rivers or migration routes
- **Polygons** – e.g. country borders or land-use zones
- **Rasters** – gridded values such as elevation or satellite imagery

Many packages exist for these data types in R: `sp`/`sf` handle vector data, `raster`/`terra` work with rasters, `stars` is specialised for spatio-temporal objects, and `spatstat` focuses on point pattern analysis, but also offers raster tools.

We will focus on sf, terra and spatstat here.

The code used for this workshop can be found under: https://github.com/InaKrapp/Workshops/tree/main. Most of the spatial datasets used in this workshop are too large to be uploaded on Github. To replicate the analysis, they have to be downloaded from their original sources.

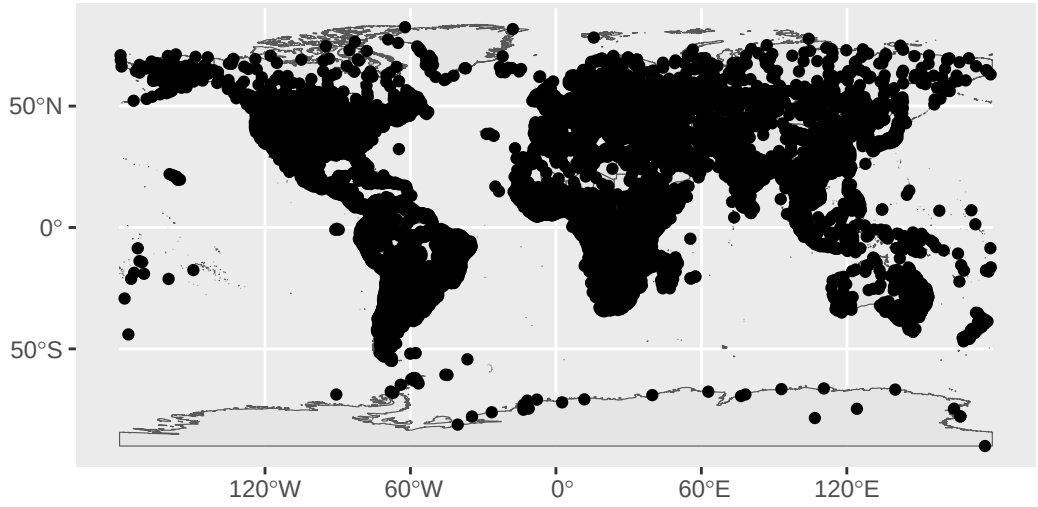
2. Get country borders and cities:

The Natural Earth project provides free vector layers that are well suited for a first look at the Korean peninsula.

Download countries and populated places from natural earth data: https://www.naturalearthdata.com/downloads/10m-cultural-vectors/

```
countries <- st_read(here("ne_10m_admin_0_countries/ne_10m_admin_0_countries.shp"))
cities <- st_read(here("ne_10m_populated_places/ne_10m_populated_places.shp"))
```

```
ggplot() +
  geom_sf(data = countries) +
  geom_sf(data = cities)
```



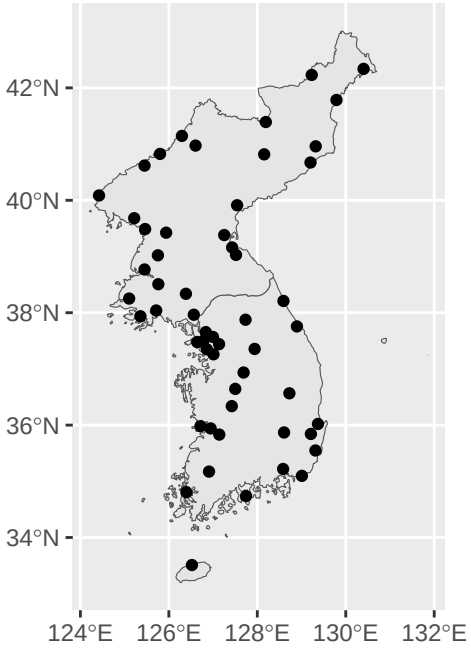
3. Select the Korean Peninsula

The `countries` and `cities` objects contain worldwide data; we extract only the parts that belong to North and South Korea.

```r
# Filter cities and countries for North and South Korea
korea_countries <- countries %>%
  filter(SOVEREIGNT %in% c("North Korea", "South Korea"))

cities_korea <- cities %>%
  filter(ADMONAME %in% c("North Korea", "South Korea"))

# Verify all korean cities lie in North or South Korea:
ggplot() +
  geom_sf(data = korea_countries) +
  geom_sf(data = cities_korea)
```



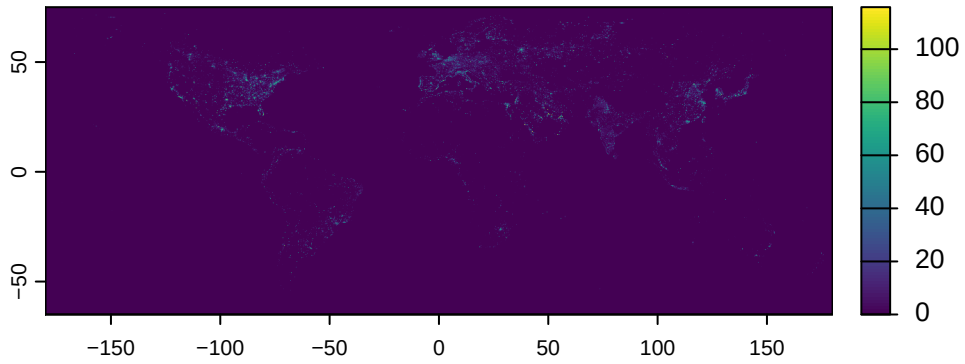
4. Load and Night-Light Data

The Consistent and Corrected Nighttime Light dataset (CCNL) from DMSP-OLS provides a global raster that records artificial illumination. You can download it here: https://zenodo.org/records/6644980

We will use the data from 2013.

This is a tif file: An image with very high resolution.

```
img <- rast("CCNL_DMSP_2013_V1.tif")
plot(img)
```

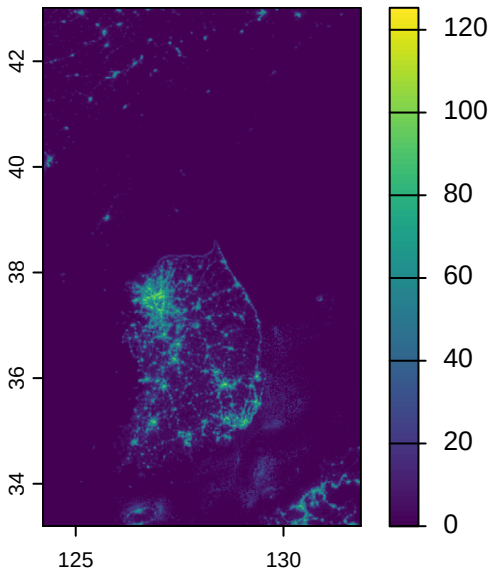


Satellite images, which covers the entire earth at high accuracy (sometimes one pixel per meter or even more detailed) can become extremly large.

terra is very effective in handling such large datasets, but it is highly advised to select only parts/areas needed for analysis before performing any operations with them. The image above, turned entirely into R's standard dataframe format, would have been over 5 GB large.

For our analysis, we only need the Korean peninsula, so we crop the data to the extent of the peninsula before any further processing.

```
# Get extent of the Korean states in a format terra understands:
v <- ext(korea_countries)
# Keep only part of the image which shows the peninsula:
img_nightlight_korea <- crop(img, v)
# Plot again to verify the new image shows the peninsula:
plot(img_nightlight_korea)
```



5. Re-project to a Projected CRS

Globally, locations are usually given in latitude/longitude. This has the disadvantage that distance calculations are difficult: One degree near the equator has a different meaning than one degree near the poles.

For local maps, projections which usually use meters/kilometers are used instead. Because they project the earth's three-dimensional surface onto a flat two-dimensional area, they are different in each region. For the Korean peninsula, we use a projection with the EPSG code 5179, which is also the official projection of the south korean government.

```
cities_korea <- cities_korea %>%
  st_transform(crs = "EPSG:5179")
korea_countries <- korea_countries %>%
  st_transform(crs = "EPSG:5179")
```

The satellite image also has to be reprojected:

```
# Reproject satellite image
img_nightlight_korea <- project(img_nightlight_korea, "EPSG:5179")
```

6. Prepare Data for `spatstat`

Unfortunately, since spatstat, sf and terra were developed with different data formats in mind, some more modifications have to be done to turn the data into formats used by spatstat:

6.1 Define the Observation Window

First, for spatstat, one needs to define a region in which the point patterns occur, called a 'window'. For our analysis, we treat only the land area of the korean peninsula as window:

```
# Create observation window: We use the union of North and South Korea
window_sf <- st_union(korea_countries)

# Convert to owin (spatstat window format)
window_owin <- as.owin(window_sf)
```

6.2 Convert City Coordinates to a `ppp` Object

In the next step, the coordinates of the korean cities are extracted and a ppp object is formed with them:

```
# Extract coordinates
city_coords <- st_coordinates(cities_korea)

# Create ppp object
city_ppp <- ppp(x = city_coords[, 1], y = city_coords[, 2], window = window_owin)
```

The cities' locations are now formed as a ppp - point pattern process. Again, we can look at the data:

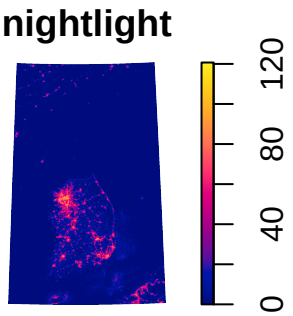
```
plot(city_ppp)
```

6.3 Convert Night-Light Raster to an `im` Object

To be able to include satellite data into the analysis, its format also needs to be adjusted:

The code below turns the satellite image into the 'im' format, which is used for raster data in spatstat.

```
# Turning it into the im format used by spatstat contains two steps:
#Turn it into a dataframe and them an 'im' object.
df <- as.data.frame(img_nightlight_korea, xy = TRUE)
nightlight <- as.im(df)
# Verify again that the data looks correct:
plot(nightlight)
```



7. Point-Pattern Modelling

Point patterns in spatstat can be modeled using a number of different models. We explore three models:

1. **Homogeneous Poisson** – assumes random locations of cities
2. **Clustered (Matérn) Process** – assumes clustering of cities
3. **Inhomogeneous Poisson with Night-Light Covariate** – assumes probability of a city location changes with light intensity

7.1 Homogeneous Poisson

The first model below assumes a poisson process: It assumes that points appear randomly with a certain probability in space. The process is called homogenous because this probability is assumed to be equal everyhwere in the window.

```
# Model 1: Homogeneous Poisson process
fit_homog <- ppm(city_ppp ~ 1)
```

7.2 Matérn Clustered Process

In reality, points often cluster and are more likely to be found close to other points. The second model assumes a matern clustering process.

This process models the idea the observed points were created as follows: First, a homogenous poisson process created points at random locations in the window. Then, points formed with an increased probability around these first points.

```
# Model 2: Clustered proess:
fit_cluster <- kppm(city_ppp ~ 1, method = "palm", clusters = "MatClust")
```

There are many other clustering processes which can be modeled.

7.3 Inhomogeneous Poisson with night light

One can also include covariates. Below is a model using the nightlight intensity from the satellite data as covariate:

```
# Model 3: Inhomogeneous with night brightness
fit_light <- ppm(city_ppp ~ nightlight)
```

7.4 Model Comparison by AIC

To evaluate the models, one can compare the AICs. They contain information about model accuracy while penalizing overly complex models.

```
# Step 5: Compare AICs
AIC_homog <- AIC(fit_homog)
AIC_cluster <- AIC(fit_cluster)
AIC_light <- AIC(fit_light)

# Display comparison
aic_comparison <- data.frame(
  Model = c("Homogeneous", "Light", "Cluster"),
  AIC = c(AIC_homog, AIC_light, AIC_cluster)
)

print(aic_comparison)
```

```
        Model      AIC
1 Homogeneous 2544.455
2       Light 2463.757
3     Cluster 8658.859
```

The model including nightlight intensity has the lowest AIC.

We can take at look at its coefficients:

```
# Print summary of best model
best_model_index <- which.min(aic_comparison$AIC)
best_model_name <- aic_comparison$Model[best_model_index]

cat("\nBest model by AIC:", best_model_name, "\n")
```

```
Best model by AIC: Light
```

```
print(get(paste0("fit_", tolower(gsub(" ", "_", best_model_name)))))
```

```
Nonstationary Poisson process
Fitted to point pattern dataset 'city_ppp'

Log intensity:  ~nightlight

Fitted trend coefficients:
 (Intercept)   nightlight
-22.99806045   0.04014719

                Estimate       S.E.      CI95.lo      CI95.hi Ztest       Zval
(Intercept) -22.99806045 0.21098933 -23.41159195 -22.58452896   *** -109.00106
nightlight    0.04014719 0.00398077   0.03234503   0.04794936   ***   10.08528
```

The night light intensity has a statistically significant effect on the probability that a city is located at a certain place.

8. A raster regression

We've seen now that light can be used to predict where to find cities. It is often used for that purpose and as a proxy variable for income. Note that the correlation 'more light = increased chance of finding a city' holds for both North and South Korea.

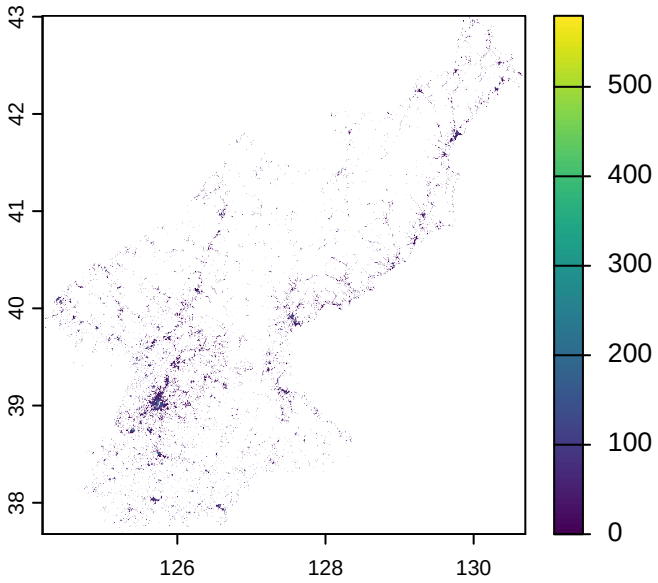
But on average, South Korea is much more brightly lit at night than the north - and this is not limited to the cities we saw on the map. We will now look at a model which models this difference in brightness.

8.1 Load Population Raster Data

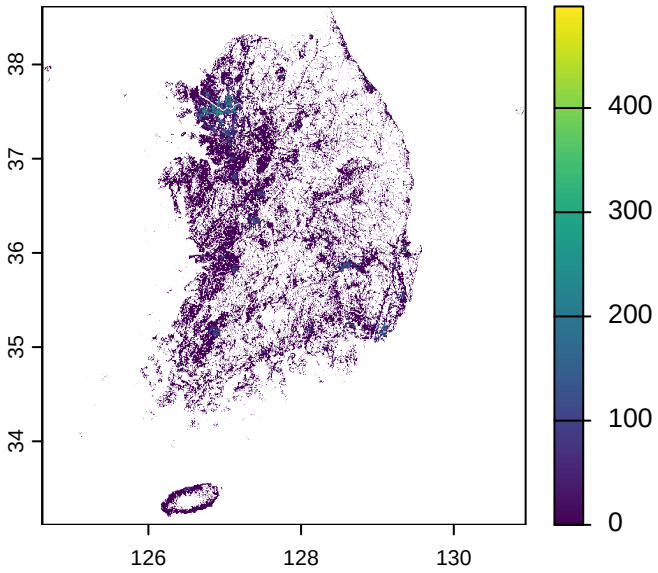
Our cities dataset does not perfectly capture where people live: While it gives the location of the peninsula's largest cities, it only contains a small number of smaller towns and villages. In point data, cities are also modeled as points while in reality, they often cover large areas. To get another view on where people live in North and Korea South Korea, we will look at another dataset.

Again, we will be working with tif images. This time, we will be using 2015 population estimates from the WorldPop project: https://hub.worldpop.org/geodata/summary?id=74005 https://hub.worldpop.org/geodata/summary?id=74021

```
img_north <- rast("prk_pop_2015_CN_100m_R2025A_v1.tif")
plot(img_north)
```




```
img_south <- rast("kor_pop_2015_CN_100m_R2025A_v1.tif")
plot(img_south)
```



```
# Re-project them to EPSG:5179
img_north <- project(img_north, "EPSG:5179")
img_south <- project(img_south, "EPSG:5179")
```

The North is generally less densely populated than the South: North Korea has around 25 Million inhabitants while around 50 Million people live in South Korea.

8.2 Build a Unified Raster Stack

In the next step, we combine the data. We now have several raster datasets, but since they are from different sources, we have to make sure they 'fit' onto each other.

```
# --- Step 1: Define the exact extent of the area our data should cover
korea_mask <- rasterize(korea_countries, img_nightlight_korea, field = 1)

# --- Step 2: Prepare the Brightness Response Variable ---
# Mask the light data and then log-transform it.
korea_lights <- mask(img_nightlight_korea, korea_mask)
final_brightness <- log(korea_lights + 0.01)
names(final_brightness) <- "log_brightness"

# 3. Create Country Predictor from sf dataset of North and South Korea borders:
korea_countries <- korea_countries %>%
  mutate(is_south = ifelse(ADMIN == "South Korea", 1, 0))
country_raster <- rasterize(korea_countries,
                                  img_nightlight_korea,
                                  field = "is_south")
final_country <- mask(country_raster, korea_mask) # Mask it to the study area.

# 4. The Population Predictor ('log_pop')
# Align each population raster to the master grid (img_nightlight_korea or korea_mask).
pop_north_aligned <- project(img_north, korea_mask, method = "bilinear")
pop_south_aligned <- project(img_south, korea_mask, method = "bilinear")

# Merge them. The result is a raster with the correct extent but NAs in the ocean.
pop_aligned_merged <- cover(pop_north_aligned, pop_south_aligned)

# Log-transform the population count:
log_pop_aligned <- log(pop_aligned_merged + 1)

# Impute all NAs with 0.
```

```r
pop_imputed <- ifel(is.na(log_pop_aligned), 0, log_pop_aligned)

# This clips away the values in the ocean, leaving only the data within the peninsula.
final_pop <- mask(pop_imputed, korea_mask)
names(final_pop) <- "log_pop"

# --- Step 5: Assemble the Final Stack and Proceed to Modeling ---
model_stack <- c(final_brightness, final_country, final_pop)
gam_df <- as.data.frame(model_stack, xy = TRUE)
```

8.3 Generalised Additive Modelling

Spatial data usually contains spatial autocorrelation (neighboring observations are correlated). There is extensive literature about possible approaches to address this issue, but no consensus.

For this course, we will use a generalized additive model, which allows to include a function that aims to predict brightness values based on x-y-coordinates.

We can fit various kinds of models such as one which only includes population, only the country (North or South Korea) or both. We will start with one which fits the s(x,y)-function that models autocorrelation.

```r
gam_model <- gam(
  log_brightness ~ s(x, y, k = 100),
  data = gam_df
)

# View the results
summary(gam_model)
```

```
Family: gaussian
Link function: identity

Formula:
log_brightness ~ s(x, y, k = 100)

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.398636   0.002083  -671.5   <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
        edf Ref.df     F p-value
s(x,y) 98.9     99 26045  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.895   Deviance explained = 89.5%
GCV = 1.3075  Scale est. = 1.307     n = 301297
```

Autocorrelation is often a powerful predictor.

But that does not mean other variables, such as the indicator if an area is north or south korean, will not be significant:

```r
gam_model_light <- gam(
  log_brightness ~ is_south + s(x, y, k = 100),
  data = gam_df
)

# View the results
summary(gam_model_light)
```

```
Family: gaussian
Link function: identity

Formula:
log_brightness ~ is_south + s(x, y, k = 100)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -2.84261    0.01263  -225.1   <2e-16 ***
is_south     3.23772    0.02794   115.9   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
         edf Ref.df    F p-value
s(x,y) 98.85     99 1382  <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =    0.9   Deviance explained =   90%
GCV = 1.2519  Scale est. = 1.2515    n = 301297
```

Areas in the South are significantly brighter in general.

Intuitively, this makes sense: Imagine you were standing at the border. Although North Korea might just be a few hundred meters away from you, if you were in South Korea, you could still expect a village on your side of the border to be much more brightly illuminated than on the other side.

Population also plays a role. Populated areas are generally brighter at night:

```r
gam_model_light_population <- gam(
  log_brightness ~ is_south + log_pop + s(x, y, k = 100),
  data = gam_df
)

# View the results
summary(gam_model_light_population)
```

```
Family: gaussian
Link function: identity

Formula:
log_brightness ~ is_south + log_pop + s(x, y, k = 100)

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -3.017897   0.012114  -249.1   <2e-16 ***
is_south     3.320496   0.026712   124.3   <2e-16 ***
log_pop      0.456764   0.002708   168.7   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
         edf Ref.df    F p-value
s(x,y) 98.84     99 1053  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
R-sq.(adj) =  0.908   Deviance explained = 90.8%
GCV = 1.1439  Scale est. = 1.1435    n = 301297
```

Again, this is what we would expect intuitively. Although South Korea's cities emit much more light during the night, in areas where no one lives, there is no need to put up streetlights.

But in North Korea, populated areas often remain dark as well. Satellite imagery of nightlights is often used as an economic indicator for areas where there is little other data available.

9. References

There are many good resources on working with spatial data in R:

An Introduction to Spatial Data Analysis and Statistics: A Course in R by Antonio Paez: https://paezha.github.io/spatial-analysis-r/ ISBN: 978-1-7778515-0-7 DOI: 10.5281/zenodo.5155982

Spatial Statistics for Data Science: Theory and Practice with R by Paula Moraga: https://www.paulamoraga.com/book-spatial/index.html

The terra package by Robert J. Hijmans has very extensive documentation online: https://rspatial.org/index.html

For more technical topics such as map projections in R, or how to work with different geodata formats, Geocomputation with R by Robin Lovelace, Jakub Nowosad and Jannes Muenchow is a good reference: https://r.geocompx.org/

The photo of the korean peninsula at night is from NASA: https://eol.jsc.nasa.gov/SearchPhotos/photo.pl?mission=ISS038&roll=E&frame=38300